

Algorithms for Data Science

Lecture #2: Running Time

Mateusz Buczyński

University of Warsaw

25.02.2026

Thanks to dr Krzysztof Fleszar for the base material.

Recap: What's your problem?!

Definition: Computational Problem

A computational problem is:

- a set of allowed inputs (instances), and
- a correct output required for each allowed input.

Input

- integer array a sorted nondecreasingly, s.t. $a[0] \leq a[1] \leq \dots \leq a[n-1]$
- integer target value v

Output

min i such that $v \leq a[i]$, or n if such index does not exist.

Example: Search Problem

Recap: Algorithm

An algorithm is a finite, unambiguous sequence of steps that:

- takes an input from a specified set of valid instances,
- produces the correct output for each valid instance,
- terminates after a finite number of steps.

Binary search

repeatedly split range in two.

Linear search

scan from left to right.

Compare algorithms?

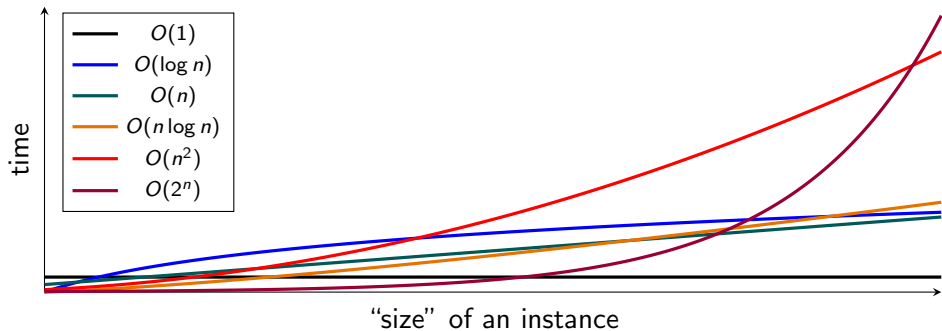
Two main questions

- How fast is the algorithm?
- How much memory does it use?

Focus of this lecture

We focus on running time today. Memory complexity is to be tackled on the next lectures.

Typical Time Complexity Growth



Time Complexity

Core idea

Running time should be analyzed as a function of input size, not as one measured runtime.

Example

For the search problem, natural input size is the array length n .

- Small n : many algorithms appear fast.
- Large n : growth rate dominates.

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best)

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;
- Average case

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;
- Average case - $\approx n/2$ iterations,
given same probability of each
instance;

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;
- Average case - $\approx n/2$ iterations,
given same probability of each
instance;
- Worst case (guaranteed upper
bound)

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) - $a[0] = v \Rightarrow 0$ iterations;
- Average case - $\approx n/2$ iterations, given same probability of each instance;
- Worst case (guaranteed upper bound) $a[n - 1] < v \Rightarrow n$ iterations.

Course convention

Unless stated otherwise, we use **worst-case running time**.

- For a fixed input size n , many instances exist.
- Worst-case picks the maximum runtime among those instances.
- This gives a robust guarantee for planning.

Slow programs

Theory

An algorithm can have poor worst-case bounds.

Practice

The same algorithm can still work very well on many real datasets.

What time measure?

1) Real time?

Use seconds / milliseconds / nanoseconds from wall-clock measurements.

Problem

Wall-clock time depends on hardware, software stack, and other technical details.

- Not universal across machines and years.
- Useful for benchmarking, not enough for theory.

What time measure?

2) Number of high-level operations?

Looks appealing, but operation costs may differ dramatically.

```
tab = [i for i in range(1000000)]  
for i in range(100000):  
    tab.pop(0)
```

```
tab = [i for i in range(1000000)]  
for i in range(100000):  
    tab.pop(len(tab)-1)
```

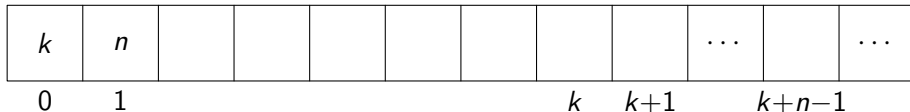
Both remove 100k elements, but runtime differs a lot due to different underlying costs.

Random Access Machine (RAM)

RAM model

A theoretical model where primitive operations have uniform unit cost.

- Hardware-independent abstraction.
- Good for asymptotic reasoning.



k - start position, n - size of array.

Random Access Machine (RAM)

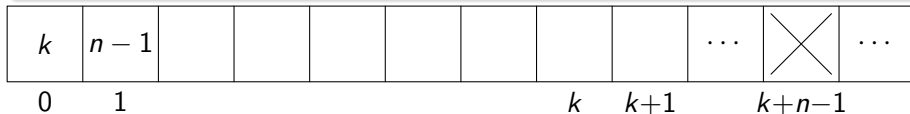
Example 1

Remove last element.

Random Access Machine (RAM)

Example 1

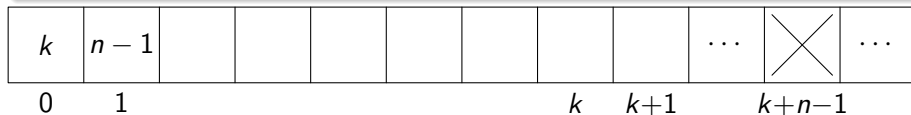
Remove last element.



Random Access Machine (RAM)

Example 1

Remove last element.



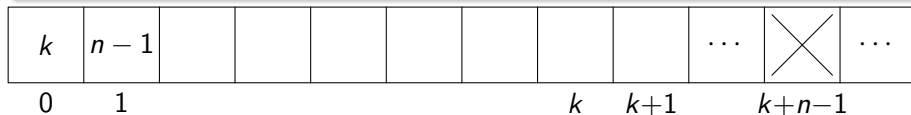
Example 2

Remove first element.

Random Access Machine (RAM)

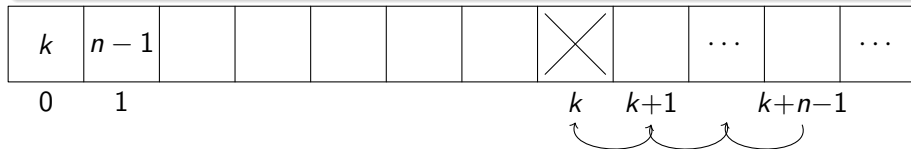
Example 1

Remove last element.



Example 2

Remove first element.



Random Access Machine (RAM)

```
1. R[3] = 0
2. R[4] = R[3] - R[1]
3. R[4] = R[4] + 1
4. IF_LARGER_ZERO R[4] JUMP_TO_LINE 11
5. R[4] = R[0] + R[3]
6. R[4] = R[R[4]] - R[2]
7. R[4] = R[4] + 1
8. IF_LARGER_ZERO R[4] JUMP_TO_LINE 11
9. R[3] = R[3] + 1
10. JUMP_TO_LINE 2
11. RETURN R[3]
```

where $R[0], R[1]$ define $\$a\$$ and $R[2]$ defines $\$v\$$.

Random Access Machine (RAM)

Why not write everything in RAM code?

It is precise, but verbose and hard to read.

- RAM-level listing can look complicated.
- You do not need to memorize this syntax.

The key message: low-level detail exists, but we need a practical analysis layer.

What time measure?

3) Number of RAM operations?

Good: proportional to real time and less tied to specific technology.

Bad: writing full RAM code is cumbersome.

Need a compromise

We want something readable like high-level code but still proportional to runtime.

What time measure?

Goal: proportional to real time

Let $f(n)$ be our analytical cost and $g(n)$ real time.

We accept bounded distortion:

$$\exists c_1, c_2 > 0 : \quad c_1 \leq \frac{g(n)}{f(n)} \leq c_2$$

for sufficiently large n .

Constants may depend on technology and implementation details.

What time measure?

Weighted high-level operations

Assign each high-level operation a weight proportional to RAM effort.

- Easy to write and read
- Simplified operation costs
- Keep proportionality to RAM cost
- Derive complexity from weighted totals

How to know operation costs?

Use lecture notes and language/library references.

What time measure?

4) Total cost of dominating operations

Start with weighted high-level costs, then simplify to a small set of dominating operations.

```
# Binary Search & Destroy
l, r = 0, n
while l < r:
    m = (l + r) // 2
    if a[m] <= v:
        r = m
    else:
        l = m + 1
if l < n:
    a.pop(l)
```

- Simplify per block.
- Then keep dominating ops only.
- Here: $\log_2 n + n$.

What time measure?

Ignore factors: asymptotics

Compare growth rates, not constant multipliers.

A: one loop with many constant-time updates

```
for i in range(0, n):  
    a += 1  
    b += 1  
    ...  
    z += 1
```

B: two loops

```
for i in range(0, n):  
    a += 1  
for i in range(0, n):  
    b += 1
```

n vs $2n$ dominating ops differ by constant factor only, so asymptotically equal.

5. Use Asymptotics Notation

Motivation

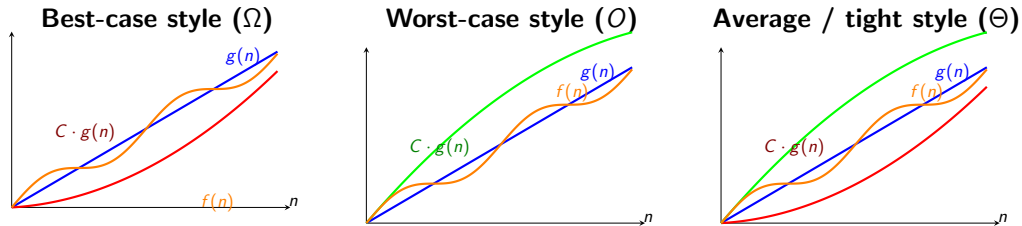
Asymptotics makes analysis and comparison simpler by focusing on dominant growth.

Examples

$$f(n) = 3 + n^2 + 1000n = O(n^2), \quad g(n) = 1000^{1000} + n^2 + \frac{1}{1000}n^3 = O(n^3)$$

- $f(n) = O(g(n))$: upper bound
- $f(n) = \Omega(g(n))$: lower bound
- $f(n) = \Theta(g(n))$: tight bound

Visual Intuition: Best, Worst, Average



Intuition only: worst-case upper bound, best-case lower bound, and tight growth band.

Asymptotic Definitions

- $f(n) = O(g(n))$

If there exist $C > 0$ and n_0 such that

$$f(n) \leq C \cdot g(n) \quad \text{for all } n \geq n_0$$

- $f(n) = \Omega(g(n))$

If $g(n) = O(f(n))$.

- $f(n) = \Theta(g(n))$

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

$$f(n) = 1000$$

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

$$f(n) = 1000$$

$$f(n) = n^2$$

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

$$f(n) = 1000$$

$$f(n) = n^2$$

$$f(n) = \frac{1}{n}$$

Typical Complexities

Polynomial complexity

$$f(n) = O(n^d) \text{ for constant } d > 0$$

Equivalent shorthand: $f(n) = n^{O(1)}$.

Dominant term rule

$$n^7 + 4n^8 + 10n^{99} = \Theta(n^{99})$$

Compare exponentials

$$n^2 = O(n^{2.5}), \quad n^2 \neq \Omega(n^{2.5}), \quad n^2 \neq \Theta(n^{2.5})$$

Typical Complexities

Exponential complexity

$$f(n) = O(d^n) \text{ for constant } d > 1$$

Compare exponentials

$$3^n \neq O(2^n), \quad 3^n = \Omega(2^n), \quad 3^n \neq \Theta(2^n)$$

Typical Complexities

More examples

$$5 = O(n^2), \quad n^2 = O(n^{10}), \quad n^{10} = O(2^n), \quad 2^n = O(3^n)$$

Hence by transitivity: $5 = O(3^n)$.

Rule (transitivity of O)

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Definitions recap: O (upper), Ω (lower), Θ (tight).

Memory complexity

- Number of memory cells
- Asymptotic notation defined analogously to running time

Examples? Next lecture!